Highlights

Hands-Free Motor Imagery EEG Classification via LLM Multi-Agents

Ruiyu Zhao, Shurui Li, Xinjie He, Xingyu Wang, Andrzej Cichocki, Yunhe Lu, Jing Jin

- We propose AutoMI, a fully automated multi-agent system for iterative MI-EEG model optimization. Orchestrating planning, execution, and output agents with specialized tools, it establishes a closed-loop framework for continuous model refinement.
- We design a hybrid decision-making mechanism coupling Q-learning with deterministic rules for action selection to circumvent unguided LLM trial-and-error, significantly enhancing optimization efficiency and stability.
- We introduce an automated co-evolution paradigm to jointly optimize hyperparameters and directly synthesize task-specific, executable structural modifications, transcending predefined search spaces.
- We construct a rule-driven feedback pipeline equipped with deterministic rollbacks and context compression to mitigate state degradation, providing robust exception-capturing and autonomous code-repair for uninterrupted, long-horizon iterations.

Hands-Free Motor Imagery EEG Classification via LLM Multi-Agents

Ruiyu Zhao[a], Shurui Li[b], Xinjie He[a], Xingyu Wang[a], Andrzej Cichocki[c,d,e], Yunhe Lu[f] and Jing Jin[a,b]

[a]The Key Laboratory of Smart Manufacturing in Energy Chemical Process, Ministry of Education, East China University of Science and Technology, Shanghai, 200237, China
[b]School of Math, East China University of Science and Technology, Shanghai, 200237, China
[c]Laboratory for Advanced Brain Signal Processing, RIKEN Brain Science Institute, Wako-shi, 351-0198, Japan
[d]Systems Research Institute, Polish Academy of Sciences, Warsaw, 01-447, Poland
[e]Department of Informatics, Nicolaus Copernicus University, Torun, 87-100, , Poland
[f] Shanghai Lansheng Brain Hospital Investment Co., Ltd. , Shanghai, 200336, China

ARTICLE INFO

Keywords:
Motor imagery
Brain-computer interface
MI-EEG classification
Multi-agent system
Automatic model optimization

ABSTRACT

Background: Motor imagery (MI) brain-computer interfaces (BCI) rely on precise electroencephalogram (EEG) classification. However, issues such as the reliance on extensive manual experience for MI-EEG model design, parameter tuning, and optimization directions, along with the poor task flexibility of foundation models and state degradation during long-term multi-agent iterations, severely restrict the state-of-the-art (SOTA) efficiency of MI-EEG. **New method**: To address these challenges, we propose AutoMI, a novel framework that uses multi-agent automated rapid iterations to construct SOTA MI-EEG models. AutoMI introduces a hybrid decision mechanism that tightly couples Q-learning strategies with deterministic rules. By integrating planning, execution, and output agents with predefined tools, AutoMI ensures broad general applicability across various hyperparameter optimizations and structural improvements. Furthermore, AutoMI integrates experience tracking and rollback mechanisms to prevent ambiguous optimization. **Results**: In evaluations on the IV2a, OpenBMI, and ECUST-MI datasets, the SOTA models finally constructed through AutoMI iterations achieve accuracies of 77.62%, 78.08%, and 83.02%, with maximum improvement reaching 24.69%, 23.35%, and 23.28% respectively. Furthermore, the average time per iteration for a single subject on the OpenBMI dataset is approximately 500 seconds. **Comparison with existing methods**: The proposed AutoMI improves accuracy by at least 5.71%, 2.92%, and 3.20% on the three datasets respectively, demonstrating both the effectiveness of the framework and the robust SOTA performance of the generated models. **Conclusion**: Experimental results indicate that AutoMI provides a novel perspective and framework design reference for future BCI model optimization.

1. Introduction

Brain-Computer Interfaces (BCIs) construct a transmission link between brain activity and computing devices. They do so by attempting to decode information from brain signals **?**. The electroencephalogram (EEG) is the dominant brain activity recording methodology used in the majority of current non-invasive BCI systems **?**. Numerous paradigms have been developed for EEG-based BCI systems based on decoding of specific neural events in the EEG. Examples of these neural events include the event-related potential (ERP) **??**, the steady-state visual evoked potential (SSVEP) **??** and Motor Imagery (MI) **??**. MI is a spontaneous BCI paradigm, requiring the BCI user to imagine the movement of body parts instead of performing actual action **?**, and is one of the most popular BCI paradigms **??**.

Over recent years, MI EEG decoding has gradually transitioned from machine learning methods primarily adopting the common spatial pattern **?** and its variants **?**, to deep learning inspired by computer vision. Various network structures, including convolutional neural networks (CNN) **?**, and Transformers**?**, are increasingly applied to the challenge of MI classification and have achieved good performance. Unlike CSP, these networks automate feature extraction and reduce reliance on manual design. With the widespread application of large-scale pretraining and agent systems, EEG classification tasks use foundation models to recognize cross-dataset challenges **??**.

Due to the inherent opacity of deep learning models and the high variance of EEG signals across subjects, structural design and hyperparameter tuning remain heavily dependent on human expertise and iterative manual tuning. Conventional optimization techniques, such as grid search or Bayesian optimization, operate strictly within predefined search spaces. This confines their scalability and precludes the integration of structural insights from existing literature. While foundation models pretrained on massive EEG datasets demonstrate strong representational capacity, they lack the generative flexibility of general-purpose LLMs to directly synthesize and refine model code. Adapting them to specific downstream MI tasks necessitates substantial fine-tuning, yet their performance frequently trails behind task-specific, state-of-the-art (SOTA) decoders **??**. Although LLM-based multi-agent frameworks circumvent the context-length constraints of single agents **?**, standard single-pass multi-agent configurations exhibit severe state degradation across successive context windows. During iterative code refinement spanning multiple rounds, the inevitable loss of

*Corresponding author
ORCID(s):
[1]Email: jinjingat@gmail.com

prior evaluation metrics, decision trajectories, and intermediate code states prevents these systems from coherently executing long-horizon optimization objectives **?**.

To tackle these specific challenges, we propose AutoMI, a fully automated multi-agent system designed for the iterative optimization of MI-EEG classification models. To overcome the limitations of iterative manual tuning and predefined search spaces, AutoMI introduces a hybrid decision-making mechanism that tightly couples a Q-learning policy with deterministic rules. Instead of relying on unguided LLM trial-and-error, this mechanism efficiently navigates the optimization space, integrates a literature retrieval tool to inject structural insights, and utilizes a rule-driven feedback node for deterministic rollbacks upon execution failures. Furthermore, rather than constructing a monolithic EEG foundation model, AutoMI leverages the generative flexibility of general-purpose LLMs to directly synthesize task-specific configuration overrides and executable structural modifications, autonomously refining specialized MI-EEG decoders. Finally, to counteract state degradation during long-horizon optimization, AutoMI establishes a resilient closed-loop framework comprising planning, execution, and output agents. Synchronized with an experience tracker that compresses historical sequences while preserving critical evaluation metrics and decision trajectories, this framework empowers the workflow with robust exception-capturing and autonomous code-repair capabilities. By seamlessly integrating these components, AutoMI establishes a stable and highly efficient optimization paradigm for BCI systems.

Evaluated on 73 subjects across three datasets, AutoMI achieves comprehensive baseline improvements strictly without human intervention. The system secures minimum classification accuracy increases of 5.71%, 2.92%, and 3.20% on the IV2a, OpenBMI, and ECUST-MI datasets, respectively, while driving a peak absolute accuracy improvement of 21.20% to reach state of the art performance. These results definitively demonstrate the capability of the AutoMI framework to autonomously optimize models and overcome the inherent limitations of manual trial and error.

The main contributions are as follows.

- We propose AutoMI, a fully automated multi-agent system for iterative MI-EEG model optimization. Orchestrating planning, execution, and output agents with specialized tools, it establishes a closed-loop framework for continuous model refinement.
- We design a hybrid decision-making mechanism coupling Q-learning with deterministic rules for action selection to circumvent unguided LLM trial-and-error, significantly enhancing optimization efficiency and stability.
- We introduce an automated co-evolution paradigm to jointly optimize hyperparameters and directly synthesize task-specific, executable structural modifications, transcending predefined search spaces.
- We construct a rule-driven feedback pipeline equipped with deterministic rollbacks and context compression to mitigate state degradation, providing robust exception-capturing and autonomous code-repair for uninterrupted, long-horizon iterations.

The rest of this paper is organized as follows. Section II surveys related work. Section III presents our proposed method, AutoMI. Section IV presents the experimental results. Section V presents a discussion of the results. Finally, Section VI describes the conclusions from our work.

2. Related work

2.1. MI Classification Model

Early MI classification employed classical machine learning algorithms such as CSP, whereas deep learning-based MI-EEG classification models further capture features through diverse structures **??**. ShallowNet utilizes a neural network to achieve log-variance calculations aimed at decoding band power features **?**. EEGNet uses depthwise separable convolution to achieve a compact structure **?**. MSFFCNN **?**, MSCNN **?** and FBMSNet **?** employs a strategy combining multiple frequency bands with multi-scale convolutions. EEGConformer **?** and CTNet **?** integrate convolutional layers with Transformer blocks.

LLMs pretrained on broad text corpora, such as Qwen, DeepSeek, and Kimi, have established strong instruction-following capabilities **???**. In EEG, large-scale pretraining typically employs dataset-specific preprocessing before applying mask-based reconstruction with transformer encoders **?**. For example, EEGPT employs masked spatio-temporal self-supervision **?**, LaBraM utilizes a vector-quantized tokenizer for masked code prediction **?**, and NeuroLM adapts LLMs via text-aligned tokenization and instruction tuning **?**. However, these models still heavily rely on manual design and hyperparameter tuning.

2.2. MI-EEG model optimization

Automated optimization of MI-EEG models has been increasingly realized through techniques such as Neural Architecture Search (NAS). For instance, MDH-NAS employs a mixed-level optimization strategy to accelerate the search process while constraining the model size for portable devices **?**. To enable broader applicability, CTNAS-EEG proposes a search space compatible with cross-task search alongside an efficient constrained search methodology **?**. Furthermore, FBNAS utilizes a framework comprising three temporal cells to process filter banks of varying frequencies, autonomously designing subject-specific network structures **?**. However, these methods are strictly constrained within predefined search spaces and suffer from prolonged search times, struggling to cope with increasingly complex MI-EEG tasks.

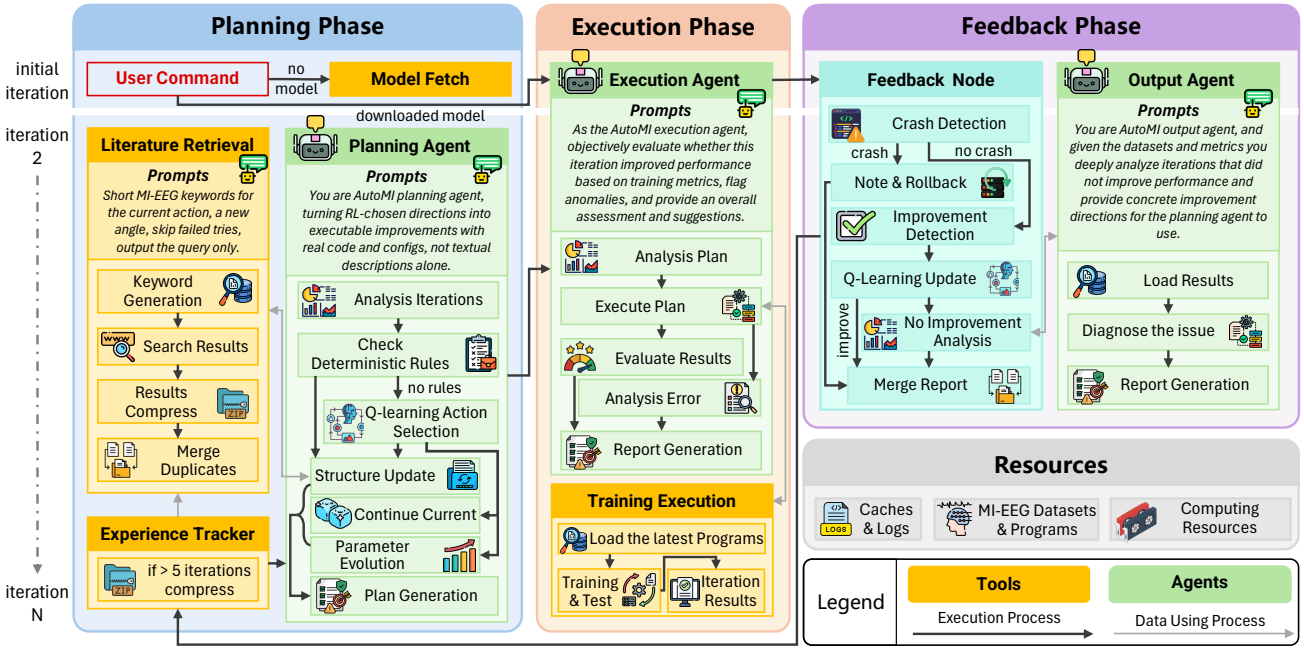

Figure 1: The framework of AutoMI.

2.3. LLM-based agent systems for EEG

LLM-based multi-agent systems transcend the limitations of single-turn inference by leveraging role specialization and collaborative protocols to solve complex, long-horizon tasks **??**. To sustain continuous execution, these systems rely on robust scaffolding and persistent state management to overcome context window constraints **?**. In the EEG, frameworks such as EEGAgent integrate planning with tool execution to automate preprocessing, feature extraction, and clinical reporting **?**. Diverging from end-to-end reporting workflows, an alternative research trajectory integrates planning and feedback mechanisms directly into iterative model training, optimizing task-specific classification performance through quantitative validation.

3. Method

This section presents AutoMI's framework and workflow as illustrated in Fig. **??**. We first establish the system formalization and define all agents, actions, and tools in Section **??**, then detail how these components are utilized across the three-phase iterative workflow of planning in Section **??**, execution in Section **??**, and feedback in Section **??**. Algorithm **??** summarizes the complete workflow integration.

3.1. Preliminaries

3.1.1. Multi-agent system

AutoMI is formally defined as a multi-agent system.

$$\mathcal{S}_{AutoMI} = \langle \mathcal{A}, \mathcal{S}, \mathcal{A}_{action} \rangle \tag{1}$$

where $\mathcal{A} = \{A_{plan}, A_{exec}, A_{output}\}$ denotes planning, execution, and output agents, $\mathcal{S}$ the state space, and $\mathcal{A}_{action}$ the action space.

Each iteration t maintains the state representation

$$s^{(t)} = (\mathbf{m}^{(t)}, \boldsymbol{\theta}^{(t)}, \mathcal{M}^{(t)}, \mathcal{H}^{(t)}) \tag{2}$$

where $\mathbf{m}^{(t)}$ is the multi-metric evaluation vector, $\boldsymbol{\theta}^{(t)}$ the hyperparameter configuration, $\mathcal{M}^{(t)}$ the current model structure, and $\mathcal{H}^{(t)}$ the historical state sequence.

3.1.2. Agent

The planning agent analyzes historical iteration results and generates executable improvement plans by selecting actions through a combination of deterministic rules and Q-learning policy, defined as:

$$A_{plan}(s^{(t)}, \mathcal{H}^{(t)}) = \begin{cases} a_{rule} & \text{if } I_{rule}^{(t)} = 1 \\ a_Q \sim \pi_\varepsilon(\cdot | s_Q^{(t)}) & \text{if } I_{rule}^{(t)} = 0 \end{cases} \tag{3}$$

where $I_{rule}^{(t)}$ is an indicator that equals 1 if the deterministic rule is triggered, and 0 otherwise. Here, a_{rule} and a_Q are the deterministic and Q-learning actions, respectively, and $s_Q^{(t)}$ represents the Q-learning state mapped from the system state.

The execution agent implements the planning agent's plans, runs model training and testing, and evaluates results with multiple metrics, defined as:

$$\mathbf{m}^{(t)} = A_{exec}(\mathcal{M}^{(t)}, \boldsymbol{\theta}^{(t)}, \mathcal{D}_{train}, \mathcal{D}_{val}) \tag{4}$$

where $\mathcal{D}_{train}$ and $\mathcal{D}_{val}$ denote the training and validation datasets, and $\mathbf{m}^{(t)}$ the multi-metric results.

The output agent diagnoses issues and provides improvement directions for the next iteration when performance stagnates, defined as:

$$\mathcal{R}^{(t)} = A_{output}(s^{(t)}, \mathcal{H}^{(t)}, \mathbf{m}^{(t)}) \tag{5}$$

3.1.3. Action Space

The planning agent operates within a discrete three-action space.

Parameter evolution keeps the network structure fixed and retunes the main training hyperparameters, defined as:

$$a_{param} : \theta^{(t)} \to \theta^{(t+1)}, \quad \theta \in \Theta \tag{6}$$

where Θ denotes the hyperparameter search space

Continue current applies narrow training-side adjustments, such as extending epochs or appending learning rate schedules, while strictly preserving the network structure and base hyperparameter configuration.

Structure update modifies the network structure under a conservative parameter constraint, defined as:

$$a_{struct} : \mathcal{M}^{(t)} \to \mathcal{M}^{(t+1)}, \quad \text{s.t. } \Omega(\mathcal{M}^{(t+1)}) \approx \Omega(\mathcal{M}^{(t)}) \tag{7}$$

where $\Omega(\mathcal{M})$ denotes the parameter count to ensure stable model complexity.

3.1.4. Tools

Four tools support the workflow.

To support structure updates, the literature retrieval tool utilizes the contextual history $\mathcal{H}^{(t)}$ to generate keywords that explore new angles and avoid past failures. It subsequently executes a sequential pipeline of search, result compression, and duplicate merging. The retrieval function is defined as:

$$\mathcal{P}^{(t)} = F_{\text{RetrieveLit}}(\mathcal{H}^{(t)}, n_{\text{p}}), \quad |\mathcal{P}^{(t)}| \le n_{\text{p}} \tag{8}$$

where $\mathcal{P}^{(t)}$ denotes the deduplicated publication set retrieved at iteration t up to a maximum count n_{p}.

The experience tracker tool maintains iteration history and provides the planning agent with compressed contextual information, defined as:

$$\mathcal{H}^{(t)}_{compress} = \begin{cases} s^{(1:t)} & \text{if } t \le n_{\text{i}} \\ s^{(t-n_{\text{i}}+1:t)} \cup \mathcal{F}_{\text{comp}}(s^{(1:t-n_{\text{i}})}) & \text{otherwise} \end{cases} \tag{9}$$

where n_{i} is the number of fully retained recent iterations, and $\mathcal{F}_{\text{comp}}$ compresses earlier states into a contextual summary.

The model fetcher tool runs in the first iteration when no local models exist and obtains MI-EEG models from a public MI-EEG classification repository.

The training execution tool load the latest model, performs model training and testing, and records evaluation metrics, defined as:

$$(\mathcal{M}^{(t)}_{trained}, \mathbf{m}^{(t)}) = \mathcal{F}_{\text{TrainExec}}(\mathcal{M}^{(t)}, \theta^{(t)}, \mathcal{D}_{train}, \mathcal{D}_{val}) \tag{10}$$

where the function returns the trained model $\mathcal{M}^{(t)}_{trained}$ and evaluation metrics $\mathbf{m}^{(t)}$.

3.2. Planning Phase

Initiated by a user command, the planning phase determines the next optimization action and generates an executable improvement plan. The first iteration establishes baseline results without Q-learning action selection. When no local MI-EEG model is present, the model fetcher tool obtains one from the public repository. The planning agent then generates an initial execution plan to train and evaluate the model.

From the second iteration, the experience tracker tool runs before each planning phase to construct the compressed contextual history $\mathcal{H}^{(t)}_{compress}$, consisting of metrics, actions, parameters, and failure analyses.

To select among the three available actions in $\mathcal{A}_{action}$, the planning agent employs a hybrid decision strategy combining deterministic rules and a Q-learning policy. Deterministic rules strictly override Q-learning. When triggered by consecutive parameter evolution failures exceeding the threshold n_{t} or execution crashes requiring structure updates, the rule executes a forced action. Otherwise, the Q-learning policy governs the action selection according to the decision function defined in Eq. **??**.

The Q-learning state space is defined as:

$$S_Q = \mathcal{L}_{metric} \times \mathcal{T}_{trend} \times \mathcal{B}_{iter} \tag{11}$$

yielding a discrete space of size $|S_Q| = 10 \times 3 \times 10 = 300$. Specifically, the metric level $\mathcal{L}_{metric}$ maps the current mean validation metric $m \in [0, 1]$ via $\max(0, \min(\lfloor 10 \cdot m \rfloor, 9))$. For the trend label $\mathcal{T}_{trend}$, let $\overline{m}$ be the mean of up to three recent metrics and set $v = 200\,(m - \overline{m})$ (where $v = 0$ for < 3 records). The downward, flat, and upward labels correspond to $v < -2$, $-2 \le v \le 2$, and $v > 2$, respectively. Lastly, the iteration bin $\mathcal{B}_{iter}$ maps the counter t via $\max(0, \min(\lfloor t/10 \rfloor, 9))$.

Action selection follows an ε-greedy policy, defined as:

$$\pi_\varepsilon(a|s_Q) = \frac{\varepsilon}{|\mathcal{A}_{action}|} + (1-\varepsilon)\mathbb{I}\big(a = \arg\max_{a'} Q(s_Q, a')\big) \tag{12}$$

where $\mathbb{I}(\cdot)$ is the indicator function that equals 1 if the condition is true and 0 otherwise.

To ensure a gradual transition from exploration to exploitation, the exploration rate decays after each Q-update, defined as:

$$\varepsilon_{t+1} = \max(0.01, 0.995 \cdot \varepsilon_t), \quad \varepsilon_0 = 1.0 \tag{13}$$

where t denotes the count of effective Q-learning updates rather than the global iteration index, ensuring exploration does not decay during execution rollbacks.

After each successful iteration, the reward r is computed based on the metric change $d = \Delta m$ between the current and previous iterations:

$$r = \begin{cases} +10, & d > 0.02, \\ +5, & 0.01 < d \le 0.02, \\ +2, & 0 < d \le 0.01, \\ 0, & d = 0, \\ -1, & -0.01 \le d < 0, \\ -3, & -0.02 \le d < -0.01, \\ -5, & d < -0.02. \end{cases} \tag{14}$$

Note that the chained thresholds structurally imply $d = -0.02$ yields $r = -5$.

Finally, the Q-values are updated with a learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.95$ **?**:

$$Q(s_Q, a) \leftarrow Q(s_Q, a) + \alpha\big(r + \gamma \max_{a'} Q(s'_Q, a') - Q(s_Q, a)\big) \tag{15}$$

For the final scheduled iteration, the bootstrap term $\max_{a'} Q(s', a')$ is strictly set to zero to handle the terminal state.

To reduce latency, the literature retrieval tool runs only when the selected action is structure update, and skips parameter evolution and continue current iterations. The tool then searches literature to obtain the publication set $\mathcal{P}^{(t)}$ according to Eq. **??**, providing reference for the planning agent.

The planning agent synthesizes the historical context, the selected action, and any retrieved literature into an executable plan. This plan provides concrete code modifications, hyperparameter configurations, and training adjustments to directly drive the subsequent execution phase.

3.3. Execution Phase

The execution agent first analyzes the improvement plan generated by the planning agent to extract model code changes, hyperparameter configuration overrides, and training strategy adjustments. Subsequently, the agent invokes the training execution tool Eq. **??** to execute the plan. Based on the prompt, the agent evaluates the test results $\mathbf{m}^{(t)}$ and, in the absence of errors, generates an execution report to guide planning decisions for the subsequent iteration. In addition, in cases where execution or training crashes due to code errors or resource issues, the agent instead generates an error report detailing the exception. Such failed executions bypass the Q-learning updates and trigger a rollback mechanism during the feedback phase.

3.4. Feedback Phase

As illustrated in Fig. **??**, the feedback phase utilizes the feedback node to evaluate outcomes via a sequential pipeline comprising crash detection, improvement detection, Q-learning update, and no improvement analysis, alongside the output agent to diagnose performance stagnation.

To prevent redundant explorations and handle exceptions within this pipeline, the feedback node applies a rule-based decision logic defined in Eq. **??**.

$$F_{\mathrm{FB}}(s^{(t)}, \mathbf{m}^{(t)}) = \begin{cases} F_{\mathrm{UQ}}(\tau^{(t)}) & \text{if } e^{(t)} = \text{success} \wedge n_{\mathrm{f}} \leq n_{\mathrm{t}} \\ F_{\mathrm{RB}}(\mathcal{M}^{(t)} \to \mathcal{M}^{(t-1)}) & \text{if } e^{(t)} = \text{crash} \\ F_{\mathrm{FS}}(s^{(t)}) & \text{if } n_{\mathrm{f}} > n_{\mathrm{t}} \end{cases} \tag{16}$$

where $e^{(t)} \in \{\text{success}, \text{crash}\}$ denotes the execution status of the current iteration. Here F_{UQ} executes the regular Q-learning update via the transition tuple $\tau^{(t)} = (s_Q^{(t-1)}, a^{(t)}, r^{(t)}, s_Q^{(t)})$. F_{RB} restores the pre-planning model version upon execution crash. F_{FS} mandates a structure update when the consecutive failure count n_{f} exceeds the threshold n_{t}.

The pipeline begins with crash detection, where any execution crash triggers the note and rollback mechanism F_{RB}. The system records the exception, strictly skips the Q-learning update, and restores the model to its prior version. The error context is then forwarded to the merge report node, forcing a structure update in the next planning phase to generate corrected code without manual intervention.

For executions without crashes, the pipeline advances to improvement detection to calculate the metric change $\Delta m = m_{acc}^{(t)} - m_{acc}^{(t-1)}$. The feedback node then executes the Q-learning update function F_{UQ} based on the corresponding reward from Eq. **??**. Specifically, it constructs the current Q-learning state and updates the Q-values using a learning rate $\alpha = 0.1$ and a discount factor $\gamma = 0.95$ according to Eq. **??**. The exploration rate ε subsequently decays following Eq. **??**.

Following the Q-learning update, the system routes the flow based on the computed metric change. If the current evaluation metrics demonstrate an improvement over the best recorded performance, the system updates the best state and proceeds directly to the merge report node. Conversely, a lack of improvement triggers the no improvement analysis branch to activate the output agent according to Eq. **??**. The output agent sequentially loads the execution results and diagnoses the issue through dataset and metric analysis. It then executes report generation to provide concrete improvement directions. Finally, the merge report node consolidates these insights to guide the subsequent planning phase.

The system employs a specialized tuning protocol when a structure update fails to yield immediate improvement. It fixes the newly generated structure and executes up to n_{t} forced parameter evolution attempts. If any parameter evolution within this window improves performance, the system retains the new structure. Conversely, if performance remains stagnant after all n_{t} attempts, the system reverts to the prior version and mandates a new structure update for the next iteration according to the rule F_{FS}. Similarly, encountering a consecutive failure count n_{f} exceeding the threshold n_{t} during normal iterations triggers the identical forced structure update rule. Before applying this forced update, the feedback node evaluates the accumulated failure count and the performance gap to determine whether to retain the current model state or restore the best recorded variant $\mathcal{M}^{(best)}$.

To conclude the iteration, the experience tracker records the final state. The system updates the historical sequence and manages the state transition according to Eq. **??**.

$$s^{(t+1)} = \begin{cases} s_{\text{init}}^{(1)} & \text{if } t = 0 \\ F_{\mathrm{Upd}}(s^{(t)}, \mathbf{m}^{(t)}) & \text{if } e^{(t)} = \text{success} \\ F_{\mathrm{RB}}(s^{(t)}, s^{(best)}) & \text{if } e^{(t)} = \text{crash} \end{cases} \tag{17}$$

This recursive function initializes the system at step zero, utilizes F_{Upd} to transition the state using evaluation metrics upon success, and executes the rollback F_{RB} upon crash. The

system then checks the iteration budget T_{max} to determine whether to initiate the next cycle or terminate.

4. Experiments

4.1. Experimental setting

We use three datasets, namely the BCI Competition IV Dataset IIa (IV2a) **?**, the OpenBMI dataset **?**, and the ECUST-MI dataset, to evaluate the iteration of AutoMI. The IV2a dataset covers left hand, right hand, both feet, and tongue classes. MI-EEG data from nine healthy participants are recorded with 22 electrodes at 250 Hz, the first session serves as training and the second as testing for each participant, and each session contains 288 trials with 72 trials per class. The OpenBMI dataset provides 62 channels EEG from 54 healthy subjects under balanced left- and right-hand MI tasks with 100 trials in both training and testing, 4s trials, and downsampling to 250 Hz. The ECUST-MI dataset is approved by the East China University of Science and Technology (ECUST-2022-054) and involves eleven healthy participants (9 males and 2 females) aged 25 to 28, of whom ten are retained after excluding one for insufficient trials. MI-EEG data are recorded from sixteen electrodes (FC5, FC1, FCz, FC2, FC6, C5, C3, C1, Cz, C2, C4, C6, CP5, CP1, CP2, and CP6) at 600 Hz for left-hand and right-hand MI tasks. Each subject completes four sessions of 30 trials each, with the first two sessions as training and the last two as testing. All trials on IV2a and OpenBMI use the 0.5–3.5 s interval. All trials of the ECUST-MI dataset use the 3 s MI cue interval.

The initial MI-EEG models adopted by AutoMI are ShallowNet **?**, EEGNet **?**, IFNet **?**, FBMSNet **?**, EEGConformer **?**, ADFCNN **?**, and CTNet **?**, and the selected LLMs are Qwen3.5 Plus **?**, Qwen3 Max **?**, DeepSeek V3.2 **?**, Kimi K2.5 **?**, GLM 5 **?**, and MiniMax M2.5 **?**, and the compared MI-EEG model optimization method is FBNAS **?**. Models refined through AutoMI iterations are evaluated based on the average accuracy of all subjects on the dataset. While each MI-EEG model is initialized with default hyperparameters, AutoMI's parameter evolution selectively optimizes these configurations in following iterations. The maximum number of AutoMI iterations is set to $T_{max} = 26$. The literature retrieval tool requests up to $n_p = 5$ publications. The experience tracker retains $n_i = 5$ recent iterations in full and compresses older history. The threshold for consecutive failures before triggering a mandatory structure update is set to $n_t = 3$. Experiments run on a NVIDIA GeForce RTX 5090 GPU, 90 GB of system RAM, and a host with 25 vCPU based on Intel Xeon Platinum 8470Q processors.

4.2. Comparison with SOTA Methods

As shown in Table **??**, AutoMI achieves SOTA performance in terms of both optimization results and optimization methods compared to SOTA models. During the evaluation on the IV2a, OpenBMI, and ECUST-MI datasets, the final SOTA models constructed by AutoMI iterations achieve accuracies of 77.62%, 78.08%, and 83.02%, with maximum improvements of 24.69%, 23.35%, and 23.28% respectively compared to SOTA methods, which proves that the MI-EEG models optimized by AutoMI can reach the SOTA level. AutoMI achieves superior performance compared to FBNAS which adopts traditional neural architecture structures, with accuracy improvements of 18.42%, 9.27%, and 19.25% on the three datasets respectively, proving the effectiveness of the automatic optimization effect of AutoMI.

Table 1
Comparisons accuracy(%) of AutoMI, SOTA models, and automated optimization method on IV2a, OpenBMI, and ECUST-MI datasets.

Model	IV2a	OpenBMI	ECUST-MI
ShallowNet	58.76	63.59	63.58
EEGNet	52.93	68.43	65.50
IFNet	69.98	71.94	69.31
FBMSNet	68.83	69.43	59.74
EEGConformer	54.44	70.26	70.03
ADFCNN	66.59	74.33	70.17
CTNet	53.16	54.73	61.82
FBNAS	59.20	68.81	63.77
AutoMI	77.62	78.08	83.02

5. Discussion

5.1. The Influence of Different LLM

Table **??** shows the impact of different LLMs in the AutoMI. Combined with Table **??**, the proposed AutoMI achieves substantial improvements across all evaluated models, and most MI-EEG models gain several to over ten percentage points after iterative optimization. During the evaluation on the IV2a, OpenBMI, and ECUST-MI datasets, the maximum improvements reach 24.69%, 23.35%, and 23.28% respectively. AutoMI automates adaptation to subject-varying EEG characteristics and performs deep tuning of complex structures. More capable LLMs tend to produce larger gains on complex structures, while the variation among LLMs remains smaller than the total improvement over the first iteration. The largest LLMs with strong code-reasoning capabilities, including Qwen3 Max **?** and DeepSeek V3.2 **?**, can effectively resolve complex implementation dependencies and therefore reach the highest performance ceilings on the most demanding structures. GLM 5 **?** reflects an aggressive optimization emphasis and can deliver strong peak gains, but it also increases the risk of unstable optimization on certain structures. In contrast, MiniMax M2.5 **?**, a mid-scale LLM, shows consistently stable optimization behavior across different base models.

Figure **??** compares the topographical maps of SOTA MI-EEG models in the initial iteration and those optimized by six LLMs. In the initial iteration, most models exhibit scattered or ambiguous activation patterns, with configurations that fail to effectively identify key channels. After AutoMI optimization, the activation patterns become more concentrated, indicating that the models learn to focus on brain regions that are truly critical for MI tasks. Different

Table 2
Mean accuracy (%) by MI-EEG model and LLM on (a) IV2a, (b) OpenBMI, and (c) ECUST-MI dataset. In each table, the highest accuracy per row is underlined, and the overall highest accuracy is highlighted in bold.

Model	Qwen3.5 Plus	Qwen3 Max	DeepSeek V3.2	Kimi K2.5	GLM 5	MiniMax M2.5
ShallowNet	66.63	67.59	66.82	66.28	67.79	66.55
EEGNet	62.89	63.08	63.73	63.35	63.50	63.39
IFNet	76.97	77.59	77.31	76.81	77.16	**77.62**
FBMSNet	75.04	74.81	74.96	75.00	74.96	75.31
EEGConformer	62.19	62.89	61.30	62.73	62.77	63.89
ADFCNN	72.38	72.45	73.53	72.30	72.49	74.00
CTNet	70.76	69.91	69.91	70.72	60.92	70.45

(a) IV2a

Model	Qwen3.5 Plus	Qwen3 Max	DeepSeek V3.2	Kimi K2.5	GLM 5	MiniMax M2.5
ShallowNet	69.39	69.19	69.41	69.59	69.31	69.78
EEGNet	71.69	71.81	71.70	72.13	72.03	71.59
IFNet	76.37	76.06	76.43	76.36	76.41	76.27
FBMSNet	73.12	73.15	71.93	72.95	73.34	72.94
EEGConformer	75.28	75.14	74.73	74.89	74.29	75.21
ADFCNN	**78.08**	77.47	77.70	77.41	77.35	77.25
CTNet	65.38	72.45	74.65	71.27	73.55	64.43

(b) OpenBMI

Model	Qwen3.5 Plus	Qwen3 Max	DeepSeek V3.2	Kimi K2.5	GLM 5	MiniMax M2.5
ShallowNet	68.48	68.98	67.62	67.46	68.47	68.13
EEGNet	70.40	71.07	71.23	71.91	70.91	70.73
IFNet	75.24	75.06	75.57	75.75	74.90	75.24
FBMSNet	67.50	66.96	66.98	66.47	67.47	67.47
EEGConformer	73.74	73.75	73.23	74.41	74.06	73.41
ADFCNN	76.08	77.25	76.10	75.58	76.09	76.09
CTNet	81.16	81.03	**83.02**	82.34	81.19	81.66

(c) ECUST-MI

models demonstrate distinct channel importance patterns after LLM optimization, proving that AutoMI performs customized optimization tailored to the characteristics of different model structures. Figure **??** shows how evaluation accuracy varies across optimization iterations for the six LLMs. For each point, the value at iteration k is the average of the best accuracy among iterations 1 through k. The traces rise more rapidly in early iterations and slow gradually later, indicating that suitable parameters or structures require multiple iterations to obtain.

5.2. The different actions of AutoMI

As shown in Figure **??**, parameter evolution is most frequently called in AutoMI because both the initial iteration and structure updates require searching for optimal hyperparameters. Structure updates are restricted in use due to the hybrid decision mechanism.

Table **??** demonstrates the decision making process of AutoMI mimicking human experts. Although the search

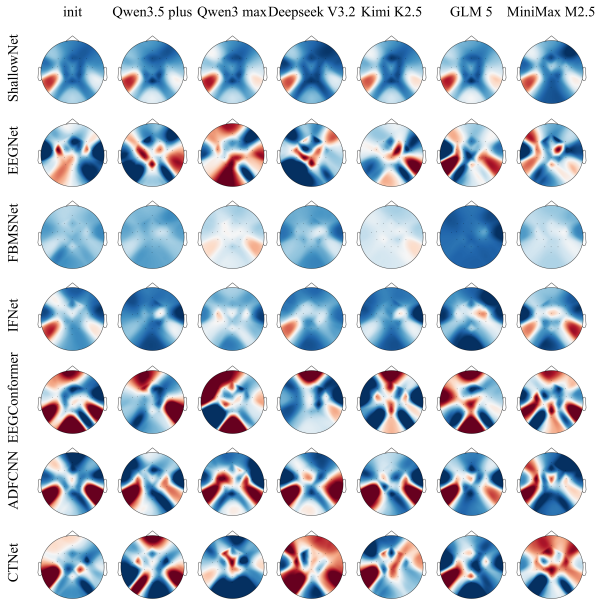

Figure 2: Topographical maps for EEG models across initial SOTA MI-EEG models and AutoMI optimized configurations in subject 1 of IV2a. The color intensity indicates the relative importance of each channel, with warmer colors representing higher importance.

results are diverse, the system can still eliminate redundancy and advance toward different structural improvements, which proves the effectiveness of the experience tracker. It can be observed that the model undergoes a rollback during the final iteration, retaining the original effective structure and proposing a new structure for optimization, which proves the validity of AutoMI.

Table **??**(a) shows that the second iteration raises the epoch budget and enables ReduceLROnPlateau, because the model reaches an accuracy bottleneck with low overfitting risk. The system extends the training epochs, dynamically adjusts the learning rate, and adds a small amount of Gaussian noise to improve generalization without altering the model structure and other hyperparameters. Moreover, Table **??**(b) shows that after utilizing hyperparameter tuning, the accuracy jumps rapidly, which proves that some subjects can achieve a good level solely through hyperparameter optimization.

5.3. Iteration Strategy

5.3.1. Optimal model feedback strategy

Figure **??** contrasts optimal model rollback counts on IV2a, OpenBMI, and ECUST-MI. OpenBMI has 54 subjects, so rollback tallies tend to exceed those on IV2a or on ECUST-MI, which retains ten subjects after screening, under the optimal model feedback strategy. On all three datasets, counts for reverting to the best accuracy checkpoint consistently exceed those for structural updates without rollback. When accuracy plateaus or declines during an experiment, rolling back to the best checkpoint provides a conservative recovery step. Extended parameter tuning

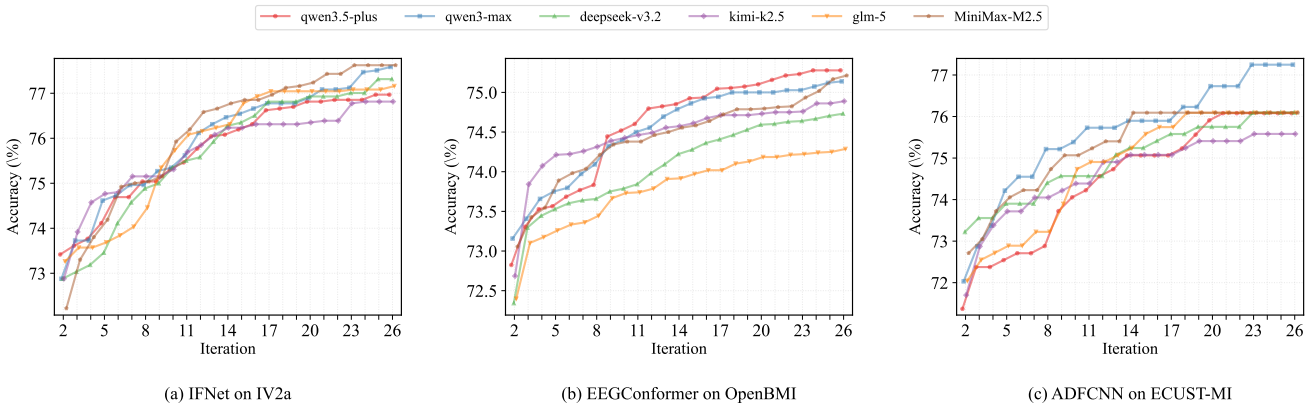

Figure 3: Mean accuracy for (a) IFNet on IV2a, (b) EEGConformer on OpenBMI, and (c) ADFCNN on ECUST-MI with six LLMs, after taking the optimal result at each iteration.

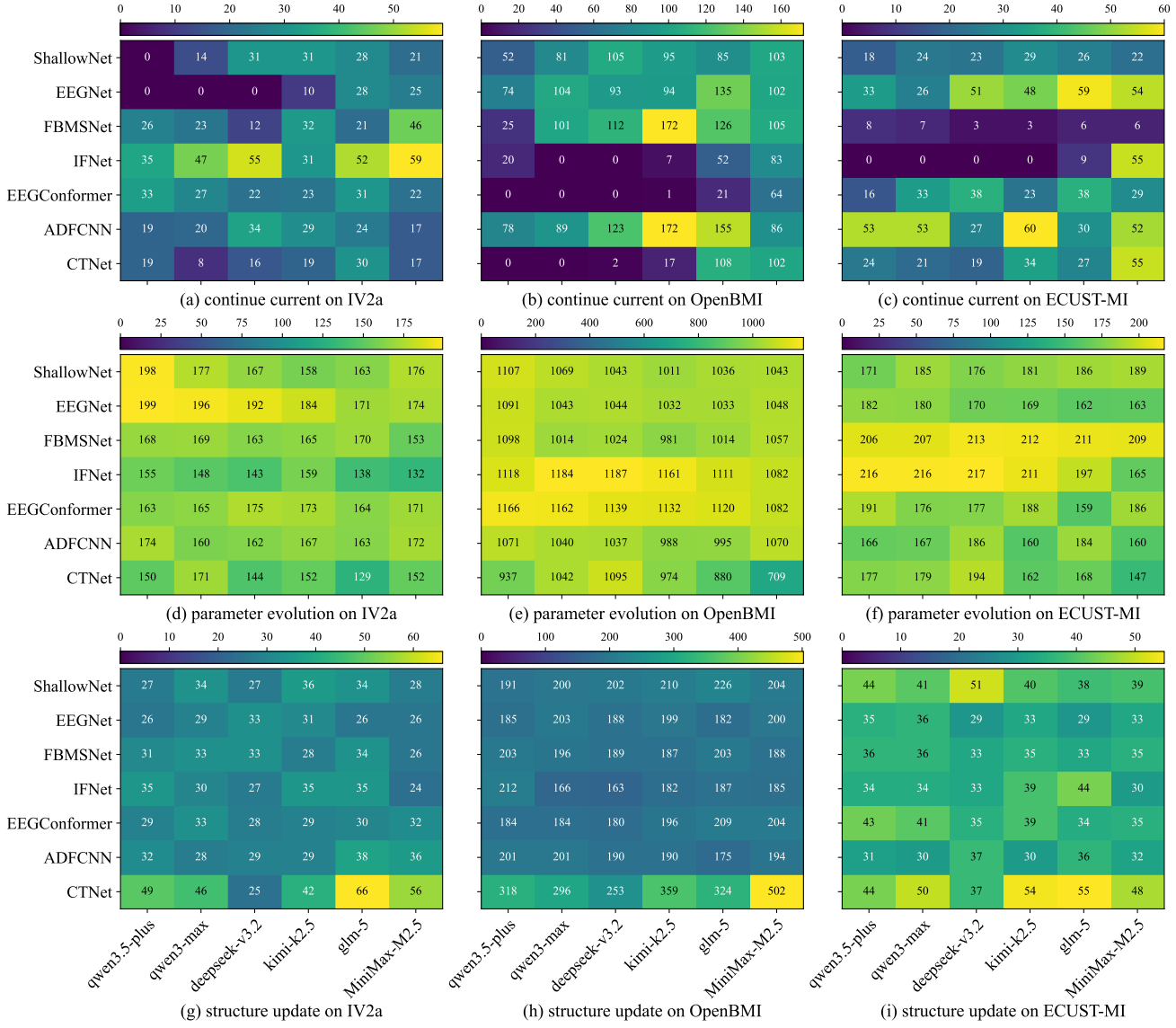

Figure 4: Counts on IV2a, OpenBMI, and ECUST-MI of three agent actions, namely continue current, parameter evolution, and structure update, when different LLMs optimize different initial MI-EEG models.

Table 3
Iteration (Iter.) results on IV2a for Subject 7 are reported for runs initialized with EEGConformer and driven by Qwen3.5-Plus, including retrieved papers, extracted content, and structural improvement measures.

Iter.	Acc. (%)	Retrieved papers	Extracted content	Improvement measures
1	51.39	No retrieve	—	No structural edit.
12	63.54	(1)**?**, (2)**?**, (3)**?**, (4)**?**, (5)**?**	(1)Lightweight MI backbones ease data hunger and latency. (2)Sinc layers stress explicit band structure and multiscale temporal cues. (3)Depthwise partial convolutions add a channel-separated spatiotemporal path. (4)Comparative MI work places SE-style gating after spatial convolutions. (5)Randomized shallow kernels with CNN–Transformer hybrids foreground robust local features and global context.	(1)SE Block, (2)CrossEntropyLoss, (3)ReduceLROnPlateau
18	51.39	(1)**?**, (2)**?**, (3)**?**, (4)**?**, (5)**?**	(1)Graph attention over electrodes and multiscale dilated temporal convolutions capture structured spatial graphs and multi-horizon dynamics. (2)MRCKT and hybrid designs repeat a noise-robust front end with transformer context. (3)Sinc parameterization targets band-aware MI spectra. (4)EEG-ITNet couples depthwise separable inception blocks with per-branch SE calibration. (5)Transfer-learning surveys favor slim backbones under limited MI data.	(1)Depthwise Separable Convolution in Spatial Filtering. (2)Update last dimension for ClassificationHead.
24	63.54	(1)**?**, (2)**?**, (3)**?**, (4)**?**, (5)**?**	(1)Comparative MI decoding benchmarks motivate channel attention after spatial filtering. (2)Mixer-style blocks reweight channelized EEG representations. (3)Transfer-learning framing supports lightweight encoders. (4)Meta-learning with depthwise separable CNNs and residuals targets rapid session or user adaptation. (5)Multiwavelet Granger causality maps motivate multiscale temporal convolutions for discriminative MI patterns.	(1)SE Channel Attention Integration, (2)last-dimension dimension correction, (3)channel size default update

Table 4
Iteration (Iter.) results list mean accuracy (Acc., %), learning rate (Lr.), training batch size (Batch), optimizer (Opt.), training epoch (Epoch), and learning-rate schedule (Sched.).

Iter.	Action	Acc.	Epoch	Sched.
1	initial test	49.65	100	—
2	continue current	57.99	300	ReduceLROnPlateau

(a) subject 5 on IV2a using IFNet with Qwen3.5 Plus

Iter.	Action	Acc.	Lr.	Batch	Opt.	Epoch
1	initial test	61.50	0.001	100	Adam	100
2	parameter evolution	98.00	0.0005	16	AdamW	300
3	parameter evolution	93.50	0.001	32	AdamW	300
4	parameter evolution	97.00	0.0006	16	AdamW	300
5	parameter evolution	98.00	0.0005	16	AdamW	300

(b) subject 21 on OpenBMI using EEGConformer with Qwen3.5 Plus

without improvement prompts subsequent structural edits, causing both counts to rise together rather than substituting for each other. When accuracy stagnates or declines, the feedback node rolls back the model to the best recorded version, while the output agent provides diagnostic analysis that informs the planning agent in its next decision. LLM-specific differences in parameter trial frequency and structural edit timing influence how often each pattern occurs.

5.3.2. Context compression

Table **??** shows how often assembled iterative context exceeds the configured threshold and triggers the context compression. Compression runs when the assembled planning text crosses a fixed length budget of 3000 characters during AutoMI iterations. OpenBMI uses the largest cohort and typically longer trajectories, so iteration logs and summaries fill the window faster than on IV2a or on ECUST-MI for a comparable depth of optimization. ECUST-MI has ten subjects, so its aggregate logging volume sits between the nine-subject IV2a setup and OpenBMI when trajectory depth is similar. Planner verbosity and structured-output habits also differ across LLMs, which makes the trigger rate depend jointly on the initial MI-EEG model, the LLM, and the dataset.

5.3.3. Iteration error analysis

Iterating with diverse hyperparameters and structures surfaces parameter errors, structure-related errors, and hardware errors. Parameter errors cover invalid hyperparameter values, incompatible configuration typing, and schema mismatches. Structure errors include GPU memory exhaustion and inconsistencies between a proposed structure and existing module boundaries. Hardware errors include unstable connectivity, LLM call timeouts, and rate limiting. Table **??** reports counts of the three error categories for each LLM, summed on IV2a, OpenBMI, and ECUST-MI and over every

Table 5
Counts of context compression events on IV2a, OpenBMI, and ECUST-MI when different MI-EEG models are paired with different LLMs. The values in each cell represent the counts for IV2a / OpenBMI / ECUST-MI datasets, respectively.

Model	Qwen3.5 Plus	Qwen3 Max	DeepSeek V3.2	Kimi K2.5	GLM 5	MiniMax M2.5
ShallowNet	20 / 139 / 31	26 / 166 / 31	23 / 146 / 29	30 / 157 / 34	20 / 152 / 29	20 / 157 / 29
EEGNet	19 / 147 / 29	20 / 144 / 25	29 / 149 / 25	22 / 154 / 32	22 / 149 / 27	19 / 167 / 32
FBMSNet	25 / 142 / 22	27 / 137 / 28	25 / 148 / 24	25 / 141 / 29	31 / 143 / 25	28 / 150 / 24
IFNet	31 / 149 / 29	31 / 124 / 29	26 / 128 / 24	33 / 124 / 23	29 / 142 / 35	24 / 128 / 25
EEGConformer	20 / 138 / 27	23 / 130 / 22	20 / 130 / 27	21 / 141 / 29	21 / 154 / 26	21 / 139 / 29
ADFCNN	26 / 145 / 27	23 / 146 / 21	24 / 146 / 30	26 / 142 / 22	24 / 138 / 25	24 / 147 / 30
CTNet	32 / 151 / 23	23 / 168 / 29	15 / 136 / 20	20 / 196 / 28	24 / 161 / 25	24 / 218 / 20

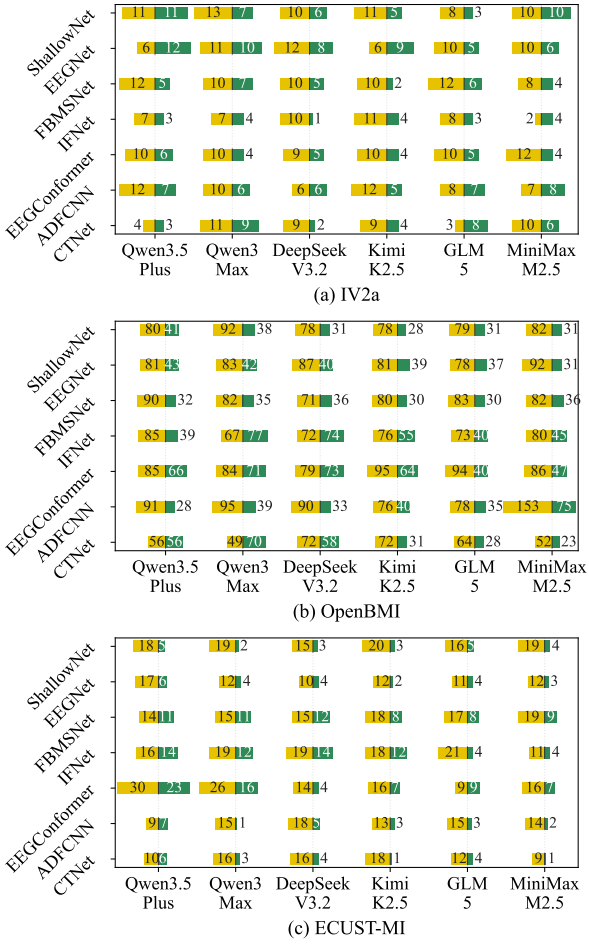

Figure 5: Optimal model feedback strategy in AutoMI using LLMs on (a) IV2a, (b) OpenBMI, and (c) ECUST-MI. *Yellow* bars to the left of each MI-EEG model tally sequences associated with reverting to the best recorded accuracy checkpoint. *Green* bars to the right tally sequences associated with forced structure updates that do not roll back to that checkpoint.

initial MI-EEG model. Structure-related errors contribute the largest column total, parameter errors the next, and hardware errors the smallest. Row totals are highest for MiniMax-M2.5 and qwen3.5-plus and lowest for deepseek-v3.2. Structural edits more often collide with GPU memory limits or with module boundaries than edits that only adjust training configuration. Wide hyperparameter search increases invalid or mistyped settings.

Table 6
Iteration error type counts for AutoMI across LLMs, reporting parameter, structure, and hardware errors summed on IV2a, OpenBMI, and ECUST-MI and every initial EEG model

LLM	Parameter	Structure	Hardware	Total
qwen3.5-plus	182	268	126	576
qwen3-max	183	222	92	497
deepseek-v3.2	150	127	115	392
kimi-k2.5	182	264	77	523
glm-5	201	300	73	574
MiniMax-M2.5	178	461	48	687
Total	1076	1642	531	3249

Table 7
Mean accuracy (%) under different ablation studies on IV2a, OpenBMI, and ECUST-MI datasets using DeepSeek V3.2 with IFNet as the initial MI-EEG model.

Ablation Component	IV2a	OpenBMI	ECUST-MI
Double Agents	72.96	73.26	71.85
Single Agent	73.57	74.42	73.05
No Experience	75.08	58.29	44.81
No Structure Update	75.73	73.22	73.71
AutoMI	77.31	76.43	75.57

5.4. Ablation study

To evaluate the rationality of the AutoMI framework, we set up different ablation experiments as shown in Table **??**. The single agent setup concentrates all functions into one agent, while the dual agent setup merges the Feedback Node in the Feedback Phase with the Planning Agent and merges the Output Agent with the Execution Agent. The setting without experience tracking disables the experience tracker. The setting without structural updates restricts actions to only parameter tuning and continuing the current state.

Our proposed AutoMI achieves the optimal result. We also observe that using a single agent is limited by context length, making its optimization direction less rapid than AutoMI. Furthermore, although adopting dual agents alleviates the context length limitation, errors in different functions and unclear stage responsibilities lead to lower accuracy than the single agent. We simultaneously find that experience

Table 8
OpenBMI computational cost for the first subject, including the mean non-training time per iteration in seconds (Infer., s), the mean MI-EEG training duration per iteration in seconds(Train., s), and the total duration of the serial campaign in hours (Total, h).

Model	Qwen3.5 Plus			Qwen3 Max			DeepSeek V3.2			Kimi K2.5			GLM 5			MiniMax M2.5		
	Infer.	Train.	Total	Infer.	Train.	Total	Infer.	Train.	Total	Infer.	Train.	Total	Infer.	Train.	Total	Infer.	Train.	Total
ShallowNet	320.7	127.3	2.7	295.9	82.2	2.6	354.0	86.3	2.4	304.5	76.2	2.5	335.8	82.1	3.0	323.8	95.4	3.0
EEGNet	333.2	95.5	2.9	301.6	112.3	2.8	328.3	137.8	3.0	304.6	146.5	3.0	313.6	90.7	2.6	345.7	107.3	2.8
FBMSNet	281.3	303.4	3.9	293.1	276.8	4.1	274.2	303.5	4.7	304.7	312.2	3.8	292.1	376.9	4.3	312.3	453.1	5.2
IFNet	306.8	309.1	4.1	297.2	146.2	2.4	310.9	141.0	2.5	272.3	96.4	2.4	286.5	112.9	2.5	279.1	113.1	2.6
EEGConformer	296.8	147.9	2.9	325.6	110.8	2.3	299.3	119.7	2.7	319.5	102.0	2.7	303.7	130.6	2.8	297.3	114.0	2.6
ADFCNN	327.3	584.2	6.0	314.7	322.0	4.3	323.7	401.4	4.7	308.2	334.0	4.4	306.9	273.3	3.5	314.2	152.5	2.9
CTNet	326.0	137.3	2.8	313.9	138.1	2.7	686.3	104.4	2.7	320.9	162.4	3.2	301.5	134.4	3.0	326.7	92.9	2.8

from previous iterations is highly crucial. Without iteration experience, the optimization direction of AutoMI becomes chaotic and fails to achieve successful iterations. Without structural updates, merely attempting parameter optimization cannot effectively compensate for structural defects, leading to a decline in results.

5.5. Computational Cost

Table **??** presents computational cost on OpenBMI for the first subject across different MI-EEG initial models and LLMs. The mean non-training time per iteration is approximately 300 seconds across pairings, while mean training duration and total hours vary more widely across MI-EEG initial models. This indicates that for simpler models with fewer parameters, such as ShallowNet and EEGNet, the LLM inference time dominates, whereas for more complex models, such as ADFCNN and FBMSNet, the training time is longer. MI-EEG initial models differ in parameter count, and the few structural updates from the LLM still do not bring final parameter counts to a common level.

6. Conclusion

To overcome the bottlenecks of exhaustive manual testing and inflexible structural design in MI-EEG decoding, we propose AutoMI, a completely automated framework that integrates large language model agents with a hybrid policy of Q-learning and deterministic rules to transcend predefined search spaces, directly synthesizing specialized architectural modifications and optimizing hyperparameters for rapid model calibration. AutoMI is evaluated on the IV2a, OpenBMI, and ECUST-MI datasets across seven baseline structures. The evaluation results show respective accuracy improvements of at least 5.71%, 2.92%, and 3.20%. These findings demonstrate that the proposed method effectively enhances the accuracy of MI tasks. Furthermore, ablation studies verify that the three-agent design aligns with practical logic for model structure improvement and parameter tuning.

Experimental results reveal that different LLMs exhibit varying evolutionary efficiencies. A stronger coding capability of an LLM leads to a higher performance upper bound. However, this difference in capability is less significant than the substantial overall improvement achieved through the initial iteration of AutoMI. During the iterative process, a rollback mechanism for flawed designs is necessary. Since the code generated by LLMs rarely succeeds on the first attempt and requires multiple debugging steps, this mechanism effectively mimics the refinement process of human experts. Furthermore, as the iteration rounds increase, the context length gradually grows. This expansion diminishes the instructional control of the prompts and requires compression techniques to resolve, thereby preventing the context from exceeding the token limits of the LLMs and maintaining memory efficiency. Regarding time costs, the inference time of the LLMs remains relatively constant, whereas the complexity of the evaluated models primarily determines the total time expenditure.

Overall, AutoMI features a multi-agent framework that mimics human experts to automatically iterate MI-EEG models. This framework provides novel perspectives and framework design references for future BCI model optimization. For future work, we plan to extend this automated multi-agent framework to various BCI paradigms.

7. Acknowledgments

This work was supported by Brain Science and Brain-like Intelligence Technology-National Science and Technology Major Project 2022ZD0208900 in part by Shanghai Municipal Science and Technology Major Project under Grant 2021SHZDZX; This research is also supported by key core technologies under Grant BE2022064-1, in part by the Lingang Laboratory under Grant No.LGL8998.